

* Introduction OR History of Python :-

python is a programming language and very simple yet very powerful object oriented programming language. The syntax of python is very simple so beginner can learn python with ease.

History of python :-

Python is developed by Guido Van Rossum. Guido van Rossum started implementing python in 1989. python is a very simple programming language so even if you are new to programming.

Features in python :-

- | | |
|----------------------------|------------------------------|
| ① Free and open source | ⑦ large standard library. |
| ② Easy to code | ⑧ dynamically type language. |
| ③ Easy to read | ⑨ Easy to debug. |
| ④ object-oriented language | ⑩ Extensible features. |
| ⑤ High level language. | ⑪ GUI programming support. |
| ⑥ Interpreted language | ⑫ Integrated language. |

python Applications :-

- ① web applications
- ② Desktop GUI Appⁿ
- ③ Console-Base Appⁿ
- ④ Software Development
- ⑤ Business Appⁿ
- ⑥ Audio-video Appⁿ
- ⑦ 3D CAD Applications
- ⑧ Enterprise Appⁿ
- ⑨ image processing
- ⑩ scientific & numeric

* Flavors of python :-

flavors of python refer to the different types of python compilers. These flavors are useful to integrate various programming languages into python.

- ① cpython
- ② Jython
- ③ Ironpython
- ④ PyPy
- ⑤ Rubypython
- ⑥ stacklesspython
- ⑦ pyjs
- ⑧ Anaconda python

* Basic syntax of python :- (Basic Function)

* print(): This function is used to print the value in python.

* Type(): This function is used to check datatype of variable.

* Input(): This function is used to take input from user at runtime.

* Datetime(): This function is used to check the date and time of system.

* Types of Comments :-

• There are 2 types of comments in python:-

① Single line comments :- # pramod

② multiline comments :- # pramod is
the developer

* Practicals :-

* Basic functions :-

* # Using print() # single line comment
print("Hello World")

output :- Hello world.

print (1 2 3)	output	1 2 3
print (0.2)	output	0.2
print (2j)	output	2j

* ## Using Type 8-

```
a = 10
print (type(a))
out: <class 'int'>
```

```
b = "pramod"
print (type(b))
out: <class 'str'>
```

```
c = 10.5
print (type(c))
out: <class "float">
```

```
d = 2+3j
print (type(d))
out: <class 'complex'>
```

* ## Using Input ()

It is used to accept the data from the keyboard which will prompt as per user data.

Ex :- per user data input ("prompt msg")

Ex ① a = input ("what is your name?")
print ("your name is =", a)

output :- your name is = pramod

② b = int(input ("what is your age?"))
print ("your age is =", b)

output :- what is your age = 22
your age is = 22

* Comments

* Data Types 8-

Data types are the classification or categorization of knowledge items. It represents the type useful that tells what operations are often performed on specific data.

① int	② float	③ complex	④ string :- char
↓	↓	↓	↓
4 bytes	4 bytes		1 byte
			1 bytes
double → 8 bytes		long double → 16 bytes	

- ① **int** :- int or integer, is a whole number positive or negative, without decimals of unlimited length.

Ex :- $x = 1$

$y = 123456789111012$

$z = -32581$

`print (type (x))` output :- `<class "int">`

`print (type (y))` output :- `—— 11 ——`

`print (type (z))` output :- `—— 11 ——`

- ② **float** :- float or floating point number, positive or negative, containing one or more decimals.

Ex :- $x = 1.10$

$y = 1.0$

$z = -35.52$

`print (type (x))` output :- `<class "float">`

`print (type (y))` output :- `—— 11 ——`

`print (type (z))` output :- `—— 11 ——`

Note :- float can also be scientific number with an 'e' to indicate the power 10.

Ex :- $x = 35e3$

$y = 12E4$

`print (type (x))` output :- `<class "float">`

`print (type (y))` output :- `—— 11 ——`

- ③ **Complex** :- Complex number are written with a "j" as the Imaginary part.

Ex :- $x = 3 + 5j$

$y = 5j$

`print (type (x))` output :- `<class "Complex">`

`print (type (y))` output :- `—— 11 ——`

④

String :-

String is collection of characters. it is represented as double or single inverted quama or string is immutable because string is not change at runtime. Index start with 0

Ex :- n = "Welcome"

print(n)	output:-	Welcome
print(len(n))	output:-	7
print(n[0])	output:-	W
print(n[-1])	output:-	e
print(n[:2])	output:-	We
print(n[3:7])	output:-	come
print(n.upper())	output:-	WELCOME
print(n.lower())	output:-	welcome
s = "Pramod"		
str1 = n + s	output:-	WelcomePramod
print(str1)	output:-	WelcomePramod
print(str1.split(" "))	output:-	"welcome" "pramod"
print d = "26-Feb-2022"		
print(d.split("-"))	output:-	"26" "Feb" "2022"

*

python operators :-

python operators are the constructs which can manipulate the value of operators. These are symbols used for the purpose of logical, arithmetic and various other operations.

Types of python Operators :-

- * Arithmetic Operators
- * Comparison (Relational) operators
- * Assignment operator
- * logical operator
- * Bitwise operator

12 0 0 7 5 2 1 0
12 2 8 4 2 1 0
32 12
66 32 12 8 5 20 6

→ * Membership Operators.

→ * Identity Operators.

→ * Arithmetic Operators :-

Python arithmetic operators are used to perform mathematical operations on numerical values.

Ex :- Addition (+), Subtraction (-), multiplication (*),
Division (/), modulus (%), Exponent (**),
Floor Division (//)

→ * Comparison Operators :-

Python comparison operators compare the value on either sides of them and the relation among them they are also called relational operators.

Ex :- Equal (==), Not Equal (!=), greater than (>),
less than (<), greater than or equal to (>=),
less than or equal to (<=)

→ * Assignment Operators :-

Python assignment operators are used to assign value to variables.

Ex :- Assignment operator (=), Addition Assignment (+=),
Subtraction Assignment (-=), multiplication Assignment (*=),
Division Assignment (/=), Remainder Assignment (%=),
Exponent Assignment (**=), floor division Assign (//=)

① → * python Bitwise operators :-

Bitwise operator works on bits and performs bit-by-bit operation.

→ * ~~Binary~~ Binary and

→ * Binary / Bitwise And (&):-

1 otherwise 0

1
1

$$1+0 = 0$$

$$0+0 = 0$$

$$0+1 = 0$$

2 * (1) Binary OR operator :-

$$\begin{array}{r} 0 \\ 0 \\ 0 \end{array} \quad \text{otherwise } 1$$

$$\begin{array}{l} 0+0=0 \\ 0+1=1 \\ 1+1=1 \\ 1+0=1 \end{array}$$

Ex All three Bitwise operator Example

$$a=60 \quad b=13$$

$$a \quad 64 \quad 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1$$

$$60 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 = 32+16+8+4=60$$

$$13 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 = 8+4+1=13$$

$$a \& b = 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 = 8+4=12 \quad a \& b = 12$$

$$a | b = 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 = 1+4+8+16+32=61$$

$$a | b = 61$$

$$a \wedge b = 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 = 32+16+1=49$$

$$a \wedge b = 49$$

3 * Binary XOR operator (XOR) :-

$$\begin{array}{r} 1 \\ 0 \\ 1 \end{array}$$

$$\begin{array}{l} 1+1=0 \\ 1+0=1 \\ 0+0=0 \\ 0+1=1 \end{array}$$

4 * (2) Binary ones Complement :-

* inverts all the bits.

$$a \sim a = -61 \quad (0100 \ 0011)$$

5 * (3) Binary left shift :-

$$a \ll 2 = 240 \quad (1111 \ 0000)$$

* shift left by pushing zeros in from the right and let the leftmost bits fall off.

6 * (4) Binary Right shift :-

$$a \gg 2 = 15 \quad (0000 \ 1111)$$

* shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off.

→ logical operator :-

operator	description	Example.
and Logical AND	if both the operands are true the condition becomes true.	(a and b) is true.

or Logical OR	if any of the two operands are non-zero then condition become true.	(a or b) is true.
------------------	---	-------------------

not Logical NOT	Used to reverse the logical state of its operand	Not (a and b) is False.
--------------------	--	-------------------------

→ membership operators :-

In	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
----	---	--

not in	Evaluates to true if it does not find a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.
--------	---	--

→ Identity Operators :-

is	Evaluate to true if the variable on either side of the operator point to the same object and false otherwise.	x is y, here is result in 1 if id(x) equals id(y).
is not	Evaluates to false if the variable on either side of the operator point to the same object and true otherwise.	x is not y here if not results in 1 if id(x) is not equal to id(y).

example practice

```
* str1 = "wel"  
print (str1, "come")  
output (welcome)
```

```
* str1 = "welcome"  
str2 = "python"  
print (str1 / str2)  
output (welcome python)
```

* using as string (%s) :-
%s is used to refer to a variable which contains a string

```
str1 = "python"  
print ("welcome %s" % str1)
```

* using other datatypes :-

* %d = integer

* %e = exponential

* %f = float

* %o = octal

* %x = hexadecimal

* %s = string

* Format statement :-

def and uses :-

The placeholder is defined using curly brackets: {}
Read more about the placeholders in the placeholder section below

* The format() ~~statement~~ method result the formatted string.

eg:-

```
string.format(value1, value2, ...)
```


* The placeholder :-

The placeholder can be identify using method indexes & price, numbered indexes {0}, or even empty placeholder {}.

Example :-

```
* print ("my name is {fname}, I am {age}").
    format(fname = "John", age = 23)
```

```
* print ("my name is {}, I am {}".format("John", 23))
```

```
* n1, n2, n3 = "111", "abc", 67.90
  print ("first {}, second {}, third {}".format(n1, n2, n3))
```

→ end Attribute :- It is used to connect which is the next print statement.

```
print ("Hi", end = " ")
```

```
print ("Hello", end = " ")
```

```
print ("How are you?")
```

output :- Hi Hello How are you ?

* Input f) :-

It is

* Type casting :-

To convert the data from one type to another type. python uses constructor function where using the same name of function data can be convert into another type.

Example :-

```
n = input ("Enter any number")
```

```
print (type(n))
```

```
print ("integer value =", int(n) + int(n))
```

```
print ("complex value =", complex(n) + complex(n))
```


Page No. _____
Date: / /

```
print ("float value", float(n) + float(n))
```

* Eval function :- Eval is used to convert the data in numerical format as per the input is evaluated.

* Write a program to accept the input through keyword and display exact datatype of it.

```
rot = eval(input("input any number"))  
print (type(rot))
```

* Write a program to accept the student name, three subject marks from the keyboard and display it using format function.

```
name = input("What is your name")  
m1 = int(input("math marks = "))  
m2 = int(input("physics marks = "))  
m3 = int(input("chem marks = "))  
print ("Student name is %s, marks of three  
subject %s, %s, %s" % format(name, m1, m2, m3))
```

* Input any number and convert it into hexadecimal or octal, float

```
a = 145  
print ("hexadecimal number = ", hex(a))  
print ("octal number = ", oct(a))  
print ("float number = ", float(a))
```


Homework 8.

- ① Input distance between two cities in km and convert into m and cm.

→ $d = \text{int}(\text{input}(\text{"distance bet"} \text{ two cities"}))$
 $dm = (d \times 1000)$
 $dcm = (d \times 100000)$
 $\text{print}(\text{"distance in m is = ", dm})$
 $\text{print}(\text{"distance in cm is = ", dcm})$

- ② input amount, Rate of interest and number of year and display simple interest.

→ $p = \text{int}(\text{input}(\text{"what is your principle amount="}))$
 $R = \text{int}(\text{input}(\text{"what is your rate of interest="}))$
 $T = \text{int}(\text{input}(\text{"what is your time="}))$
 $\text{Simple interest} = (p \times R \times T) / 100$
 $\text{print}(\text{"simple interest is = ", simple interest})$

- ③ Input any ~~number~~ two number and swap it.

$a = \text{int}(\text{input}(\text{"first number="}))$
 $b = \text{int}(\text{input}(\text{"second number="}))$
 $t = a$
 $a = b$
 $b = t$
 $\text{print}(\text{"first swap value", a})$
 $\text{print}(\text{"second swap value", b})$

- ④ Input Basic salary of an employ and calculate Bonus which is 22% of Basic salary.

→ $A = \text{int}(\text{input}(\text{"what is your salary"}))$
 $\text{Bonus} = (A \times 22 / 100)$
 $\text{print}(\text{"Bonus of employ is = ", Bonus})$

* Control statement :-

To keep a control on execution of a program control statements are used for decision making we use control statements.

① if statement :- It is used to only two part of the condition if the condition is false then code get skip.

Syntax :- `if (condition) :-
statement`

Ex :- `Age = int(input("Age of the person"))
if (age > 18):
 print("Adult")`

② if else statement :- It is used to check only true part of condition and else is used to check only false part of if condition.

Syntax :- `if (condition) :-
statement
else:
statement`

③ Ex :- `Age = int(input("Age of the person"))
if (Age > 18):
 print("Adult")
else:
 print("Not Adult")`

Ex :- `username = input("What is your username")
password = input("What is your password")
if (len(username) == len(password) and password.isalnum()):
 print("Successful login")`

else :

print ("username & password wrong")

* write a program to input any alphabet and check it is vowel or not.

```
→ ch = input("Any alphabet = ")
if (ch == "a" or ch == "e" or ch == "i" or ch == "o"
    or ch == "u") :
    print ("It is vowel")
else :
    print ("It is not vowel")
```

* Input Weight and age

```
Weight = int(input("what is your weight"))
```

```
Age = int(input("what is your age"))
```

```
if (Weight > 50 and Age > 18) :
```

```
    print ("your are eligible")
```

```
else :
```

```
    print ("your not eligible")
```

* Pid = int(input("patient ID = "))

```
if (len(str(pid)) == 8) :
```

```
    print ("valid patient ID")
```

```
else :
```

```
    print ("not valid ID")
```

* Input Current year and year of joining if the year of service is more than 3 year then bonus of Rs 2500 will issues either no bonus.

```
→ C-year = int(input("current year"))
```

```
J-year = int(input("joining year"))
```

```
if (C-year - J-year > 3) :
```

```
    print ("Bonus of 2500")
```

```
else :
```

```
    print ("No Bonus")
```


IF - elif - else statement :-
To check multiple condition if elif statements is used.

Syntax :- if (condition):
 statement

 elif (condition):

 statement

 else:

 statement

* Accept any number through the keyword and test it is positive, negative or zero.

```
num = int(input("Enter any number"))
```

```
if (num > 0):
```

```
    print("positive")
```

```
elif (num < 0):
```

```
    print("negative")
```

```
else:
```

```
    print("zero")
```

* input any value and check it is integer, float, complex, string or invalid.

```
→ num = eval(input("Enter any value"))
```

```
if (type(num) == int):
```

```
    print("Integer type value")
```

```
elif (type(num) == float):
```

```
    print("Float type value")
```

```
elif (type(num) == complex):
```

```
    print("Complex type value")
```

```
elif (type(num) == str):
```

```
    print("String type value")
```

```
else:
```

```
    print("Invalid")
```


* If bill exceeds rupees 400 then search charge of 50. it will be charge and the minimum bill should be of rupees 1000.

```
→ CN = input("customer name")
CID = input("customer id")
VCC = int(input("unit consumed by cost"))
charge = net = bill = 5 = 0
if (VCC > 200 and VCC <= 399):
    print("charge is 1.50rupees per unit")
elif (VCC >= 400 and VCC <= 599):
    print("charge is 1.80rupees per unit")
else:
    print("charge is 2per unit")
```

* input any string value and check it is upper case, lower case, or numeric.

```
→ num = input("what is your string value")
if (num == num.upper()):
    print("value is upper")
elif (num == num.lower()):
    print("value is lower")
else:
    print("value is another type")
```

* Input any three number and find which is smaller than them.

```
→ a = int(input("first number="))
b = int(input("second number="))
c = int(input("third number="))
if (a < b and a < c):
    print("a is a smallest number")
elif (b < a and b < c):
    print("b is a smallest number")
elif (c < a and c < b):
    print("c is a smallest number")
```


else:

print("something went wrong")

* Input three side of a triangle and check whether it is equilateral, scalan or isoscan.

→ $x = \text{int}(\text{input}("x:"))$

$y = \text{int}(\text{input}("y:"))$

$z = \text{int}(\text{input}("z:"))$

if $(x = y = z)$:

print("Equilateral triangle")

elif $(x \neq y \text{ or } y \neq z \text{ or } z \neq x)$:

print("isoscalen Triangle")

else:

print("scalene triangle")

* A person wants to buy a ticket for football match if the age of person is the above 11 then eligible to watch a match. The ticket cost for 16 to 50 age group is 2500 and either 1500.

→ $\text{Age} = \text{int}(\text{input}("what is your age"))$

if $(\text{age} \leq 11)$:

print("eligible to watch match")

elif $(\text{age} < 11)$:

print("not eligible to watch match")

elif $(\text{age} \geq 16 \text{ and } \text{age} \leq 50)$:

print("ticket price is 2500")

elif $(\text{age} > 50)$:

print("ticket price is 1500")

else:

print("your not eligible")

* Loops :-

looping statement are used for iteration purpose. The purpose of the loop is to repeat the same code a number of times.

* python support two types of looping statement :-

(1) while loop

(2) for loop.

(1) while loop :- while is a most important loop for iteration purpose.

Syntax :- while (test condition) :

Statement

counter:

Ex :-

```
i = 1  
while (i <= 10):
```

```
    print(i)
```

```
    i = i + 1
```

```
print("loop finished")
```

* Print table of 5 that is 5 to 50

→ i = 5

```
while (i <= 50):
```

```
    print(i)
```

```
    i = i + 5
```

i = 0

```
while (i <= 10):
```

```
    print(i * 5)
```

```
    i = i + 1
```

* input any number and print its table.

→ i = 1 num = int(input("what is a number"))

```
while (i <= 10):
```

```
    print(num * i)
```

```
    i = i + 1
```


* Input any number and print square value of each from 1

→ num = int(input("Enter any number"))

i = 1

while (i <= num):

print(i, " = " i * i)

i += 1

* input any number and find its factorial value.

→ F = 1

num = int(input("Enter any num"))

i = 1

while (i <= num):

F = F * i;

print("factorial value = ", F)

i += 1

* print divisible of 5 and 7 between 1500 to 2700

n = 1500

while (n <= 2700):

if (n % 5 == 0 and n % 7 == 0):

print(n)

n += 1

* input any string and print reverse value.

→ s = ""

s = input("Enter any string")

index = len(s)

while [index > 0]:

s += s[index - 1]

index -= 1

print("reverse string", s)

primma
3
5 7 0
s += s (s-1)
index - 1

* input any number and check it is palindrome or not
 → num = int(input("Enter any Number"))
 temp = num
 rev = 0
 while (num > 0):
 dig = num % 10
 rev = rev * 10 + dig
 num = num // 10
 if (temp == rev):
 print("The number is palindrome")
 else:
 print("number is not palindrome")

* input any number and check it is armstrong and not
 → num = int(input("Enter a number"))
 Sum = 0
 temp = num
 while temp > 0:
 digit = temp % 10
 Sum += digit ** 3
 temp // 10
 if (num == Sum):
 print(num, "is an Armstrong number")
 else:
 print(num, "is not armstrong number")

* for loop :- It is most important looping statement which is useful to traverse the data structure like list, set, tuple, dictionary.

Ex :- for loop using sequence :-

```
str = "python"
for i in str:
    print(i)
```

output :-

p
y
t
h
o
n

* program to print the table of the given number
list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

→ n = 5

```
for i in list:
    c = n * i
    print(c)
```

* input any alphabet and print it is a vowel or not using for loop.

① alpha = ['a', 'e', 'i', 'o', 'u']

ch = input("Enter any alphabet")

count = 0

for i in alpha:

if i in ch:

count += 1

print("Total vowel", count)

② text = "deep learning"

count = 0

vowel = ['a', 'e', 'i', 'o', 'u']

for i in text:


```
if i in vowel:
```

```
    count += 1
```

```
print("Total vowel", count)
```

* write a program to print 1 to 10 number except 3, 6, 8 using continue statement.

```
→ for n in range(1, 11):
```

```
    if (n == 3 or n == 6 or n == 8):
```

```
        continue
```

```
    print(n)
```

* print square value of 1 to 10 using for loop.

```
→ for i in range(1, 11):
```

```
    c = i * 2
```

```
    print(c)
```

* print the sum of given number

```
list = [10, 30, 23, 43, 65, 12]
```

```
sum = 0
```

```
for i in list:
```

```
    sum = sum + i
```

```
print("The sum is :", sum)
```

* for i in range(10):

```
    print(i, end=" ")
```

output :- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

* Table of given number

```
n = int(input("Enter the number"))
```

```
for i in range(1, 11)
```

```
    c = i * n
```

```
    print(n, "*", i, "=", i)
```


~~print (random value)~~

str = input ("Enter any number/world :")

i = 0

for i in range (len (str)):

if str [i] != str [-1-i]:

print ("It is not a palindrom")

breaks

else :

print ("It is a palindrom")

* print Armstrong number between 100 to ~~1000~~ 200

for i in range (100, 201):

pow = len (str (i))

Sum = 0

temp = i

while temp > 0 :

digits = temp % 10

Sum += digits ** pow

temp //= 10

if i == Sum :

~~print~~ print (i)

* text = ["xyz", "pqr", "12", 'a', "@123@", "99"]

c = 0

for i in text :

if (len (i) > 2 and i[0] == i[-1]):

c += 1

print ("total count = ", c)

* write a program to input any number and check the number is prime or not.

```

→ num = int(input("Enter any number"))
if (num == 1):
    print("non prime")
elif (num == 2):
    print("It is prime")
else:
    n = num // 2
    for i in range(2, n+1):
        if (num % i == 0):
            print("non prime")
            break
    print("It is prime")

```

* write a program to print armstrong number betⁿ 100 to 199 using for loop [Armstrong no. eg $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$]

```

→ for i in range(100, 1000):
    id = i // 100
    sd = (i - id * 100) // 10
    td = i % 10
    if (i == id**3 + sd**3 + td**3):
        print("Armstrong number")

```

* nasted for loop :-

It is used to print the data in the form of row and column.

It is also used to print table or matrix in form of data.

row → for var in sequence:

column → for var in sequence.

i == row

j == column

* write a program to print the pattern.

```

*
* *
* * *
* * * *

```

```

→ for i in range(4):
    for j in range(i+1):
        print("*", end=" ")
    print()

```

* print following pattern

```

* * * *
* * *
* *
*

```

```

→ for i in range(4):
    for j in range(4, i, -1):
        print("*", end=" ")
    print()

```

* print the following pattern.

```

1
1 2
1 2 3
1 2 3 4

```

```

→ for i in range(1, 5):
    for j in range(1, i+1):
        print(j, end=" ")
    print()

```

```

* for i in range(1, 5):
    for j in range(i, 0, -1):
        print(j, end=" ")
    print()

```

```

1
2 1
3 2 1
4 3 2 1

```


→ 6 5 4 3 2 1
 6 5 4 3 2 for i in range (6):
 6 5 4 3 for j in range (6, i, -1):
 6 5 4 print(j, end = "\t")
 6 5 print()
 6

* 5 4 3 write a program to print pair of
 pythagorus from 1 to 50:
 → for i in range (1, 51):
 for j in range (1, 51):
 for k in range (1, 51):
 if (k² = i² + j²):
 print(i, "\t", j, "\t", k)

* *
 * * *
 * * * *
 * * *
 * *
 *

* Python List :-

list is a collection of python, it is represent of square brackets []. list is a mutable. we can perform the insert, delete, update at a run time. and it has inserted multiple datatype value.

A list is collection of dissimilar data element where it is can store duplicate order and index from of element.

Ex :-

```
* thislist = ['apple', 'banana', 'cherry']  
print (thislist) print (type(thislist))  
output : ['apple', 'banana', 'cherry']  
: < class 'List'>
```

```
* thislist = ["apple", 'banana', 'mango']  
print (thislist [1])  
output : ['banana']
```

```
* thislist = ["apple", 'banana', 'cherry']  
thislist [1] = "blackcurrent"  
print (thislist)  
output : ['apple', 'blackcurrent', 'cherry']
```

```
* thislist = ['apple', 'banana', 'cherry']  
for x in thislist :  
    print (x)  
output : ['apple', 'banana', 'cherry']
```

```
* Check if "apple" is present in the list  
thislist = ['apple', 'banana', 'cherry']  
if 'apple' in thislist :  
    print ("yes, 'apple' is the fruits list")  
output : yes, apple is the fruits list
```

```
* thislist = ['apple', 'banana', 'cherry']  
print (len (thislist))  
output :- 3
```

```
* thislist = ["apple", 'banana', 'cherry']  
thislist .append ("orange")  
print (thislist)  
output :- ['apple', 'banana', 'cherry', 'orange']
```


* `thislist = ['apple', 'banana', 'cherry']`
`thislist.insert(1, 'orange')`
`print(thislist)`
output :- `['apple', 'orange', 'banana', 'cherry']`

* `thislist = ["apple", 'banana', 'cherry']`
`thislist.remove("banana")`
`print(thislist)`
output : `["apple", "cherry"]`

* Pop :-
`thislist = ['apple', 'banana', 'cherry']`
`thislist.pop()`
`print(thislist)`
output : `['apple', 'banana']`
* `thislist.pop(1)`
output : `['apple', 'cherry']`

* delete :-
del Remove the specific index :-
`thislist = ['apple', 'banana', 'cherry']`
`del thislist[0]`
`print(thislist)`
output : `['banana', 'cherry']`

* del keyword can also delete the list completely.
`thislist = ['apple', 'banana', 'cherry']`
`del thislist`
`print(thislist)` # `thislist` is permanently deleted.

* clear () :-
The clear method empties the list
`thislist = ['apple', 'banana', 'cherry']`
`thislist.clear()`
`print(thislist)`

* The list() constructor :-
It is also possible to use the list() constructor to make a list.

Ex. `thislist = list('apple', 'banana', 'cherry')`
`print (thislist)`
output :- `['apple', 'banana', 'cherry']`

* Join two list :-
There are several way to join or concatenate, two or more list in python.

one of the esire ways are by using the `+` operator

Ex.

`list1 = ['a', 'b', 'c']`
`list2 = [1, 2, 3]`
for `x` in `list2` :-
`list1.append(x)`
`print (list1)`

join 2 list :-

`list1 = ['a', 'b', 'c']`

`list2 = [1, 2, 3]`

`list3 = list1 + list2`

`print (list3)`

* Extend :-

`list1 = ['a', 'b', 'c']`

`list2 = [1, 2, 3]`

`list1.extend(list2)`

`print (list1)`

output :- `['a', 'b', 'c', 'd', 1, 2, 3]`

* Copy :-

`thislist = ['apple', 'banana', 'cherry']`

`mylist = thislist.copy()`

`print (mylist)`

output :- `['apple', 'banana', 'cherry']`

* Create a list for 1 to 20 element and print squar of a list except 1st 5

```
l1 = [1, 2]
```

```
sqr = []
```

```
for i in range(11):
```

```
    squar.append(i + i)
```

```
print("This is squar no", squar[5:])
```

→ write a program to create a list and accept the input for keyboard as per the given element.

```
→ list = []
```

```
# or list = list()
```

```
print("Data type", type(list))
```

```
n = int(input("Enter number of element"))
```

```
for i in range(0, n):
```

```
    list.append(eval(input("Enter new element")))
```

```
print(list)
```

* create a list and find max value between them

```
list1 = [12, 34, 56, -78, 90, 3, 5, 2]
```

```
max = 0
```

```
for a in list1:
```

```
    if a > max:
```

```
        max = a
```

```
print(max)
```

→ write a program to check given list is empty or not.

```
→ l = [1, 3.4, 'fgh']
```

```
if not l:
```

```
    print("list is empty")
```

```
else:
```

```
    print("list is not empty")
```


* Thislist = ['apple', 'banana', 'cherry', 'mango', 67,
 89.78, 'lewi']
 print (thislist [1:2])
 print (thislist [::-1])
 print (thislist.reverse())
 print (thislist [:3])
 print (thislist [2:6])

* write a python program to remove duplicate or
 unique from a list.

→ duplicate = [2, 4, 'Abc', -34, 'xyz', 'Abxx', 20, 5,
 'Abc', 2, 20, 4]

final_list = list()

for num in duplicate:

if num not in final_list:

final_list.append(num)

print (final_list)

* write a program to print sum of 11 to 30 element
 using list.

→ num = range (11, 31)

sum = 0

for i in num:

sum += i

print ("Addition =", sum)

* Create list of color and print only list of those which contains 0

→ color = ['yellow', 'pink', 'orange', 'Blue']

check = list()

for i in color:

if '0' in i:

check.append(i)

print ("check list =", check)

* Create a list of elements which may carry numericals only and print the list which contains divisible of 5

→ list = [2, 5, 8, 10, 13, 15, 17, 20]

for i in list:

if i % 5 == 0:

print(i)

* list = [2, 5, 8, 10, 13, 15, 17, 20]

check = list()

for i in list:

if i % 5 == 0:

check.append(i)

print("check list =", check)

* list = [1, 2, 2, 3, 3, 4, 5]

unique = list(set(list))

print(unique)

output: [1, 2, 3, 4, 5]

* x = [1, 2, 3, 4]

f = [1, 1, 2, 3, 4, 9]

for element in f:

if element in x:

x.remove(element)

print(element)

OR unique = list(set(x+f))
print(unique)

* Nested list :-

l1 = [1, 2, [4.5, 'Abc', [2.5, 75], 39], 99, 'x4']

print(l1)

print(l1[1]) → [2]

print(l1[2]) → [4.5, 'Abc', [2.5, 75], 39]

print(l1[2][0]) → [4.5]

print(l1[2][2][1]) → [7.5]

print(l1[2][3]) → [39]

* List Comprehension:-

it offers shortest syntax when you want to create a new list based on the value of existing list.

* Write a program to create a list of fruits name and store the list element which may contains alphabet 'a' only.

```
fruits = ['mango', 'cherry', 'Apple', 'kiwi']
newlist = [x for x in fruits if 'a' in x]
print (newlist)
```

```
* fruits = ['mango', 'cherry', 'Apple', 'kiwi']
newlist = [x for x in fruits]
print (newlist)
test = [x for x in fruits if x != "apple"]
print (test).
```

```
* fruitlist = ['mango', 'cherry', 'banana', 'kiwi']
newlist = [x for x in fruitlist if 'a' in x]
print (newlist)
l1 = [x if x != "banana" else "orange" for x in fruitlist]
print (l1)
```

```
* l1 = range (1, 21)
even = [x for x in l1 if x % 2 == 0]
print ("Even no list", even)
t = ["Even" if x % 2 == 0 else "odd" for x in range (1, 21)]
print (t).
```

```
* l1 = range (1, 31)
test = [x for x in l1 if x % 2 == 0 if x % 5 == 0]
print (test).
```


* Tuple Tuple :-

Tuple is collection of python. It is represented as round bracket(). Tuple is immutable. It is not performed insert, delete, update at runtime.

Tuple has inserted/stored multiple datatypes.

Ex. `tuple = ('A', 23, 'x7', 4.5, 5.8)`

```
* t1 = ('A', 23, 'x7', 4.5, 5.8)
print(t1)
for i in t1:
    print(i)
print(t1[3])
print(len(t1))
```

* Tuple Operations :-

```
t1 = (1, 2, 3, 4)
t2 = (21, 22, 23)
print(t1 + t2)
t3 = t1 + (99, 88, 77) + t2
print(t3)
```

* Create a tuple and input the specific value and delete them as per the user choice.

```
→ student = tuple()
st = list(student)
n = int(input("How many student?"))
for i in range(n):
    st.append(input("Enter new element"))
print("Tuple element", st)
print(type(st))
st.pop()
student = tuple(st)
print(type(student))
```


* Set :- A set is a collection of unique data. That is, elements of a set cannot be duplicate. ~~for~~, it is represented as a curly brackets {}.

```
* emp = {'abc', 'x42', 'abc', 'pqr'}
print (emp)
print (type (emp))
emp.add ('testing')
print (emp)
emp.update (['A', 245, 3, 5])
print (emp)
emp.remove (245)
print (emp)
emp.discard (45)
print (emp)
print ("length = ", len (emp)).
```

* write a program to find max value and min value from the given set of element.

```
n1 = {11, 21, 45, 78, 99, -3, 908}
print ("set element", n1)
print ("max value of set =", max(n1))
print ("min value of set =", min(n1))
```

* Frozen set :-

it is an immutable version of a python set object. While element of python can be modified at any time element of frozen type remain same after creation. Also this frozen set can be used as keys in dictionary or as elements of another set. But like sets it is not ordered. This funⁿ set single parameter.

* Frozen set operation :-

like normal sets frozen set can also be perform different operations like copy, intersection, difference & union.

* create two different set and display their intersection and union in python.

```
→ A = { 1, 2, 3, 4 }
   B = { 3, 4, 5, 6 }
   C = A.intersection(B)
   print ("Intersection", C)
   D = A.union(B)
   print ("Union", D)
```

* Dictionary :- Dictionary is the collection of python. It is represented as curly brackets {}. It is the combination of key and value. It has key immutable and value is mutable.

Ex:-

```
d1 = { 'Red': 'Apple', 'Green': 'grass', 'Blue': 'Berry' }
print(d1)
for i in d1:
    print(i)
for i, j in d1.items():
    print(i, "=", j)
for x in d1.keys():
    print(x)
for y in d1.values():
    print(y)
```

Ex Cricket = dict()

```
n = int(input("How many players?"))
for i in range(n):
```

```
    k = input("player name")
```

```
    r = eval(input("score of player"))
```

```
    (Cricket.update({ k:r }))
```

```
print("Cricket player", Cricket)
```

```
for i in Cricket.keys():
```



```

print(i)
player = input("player name:")
run = cricket.get(player, -1)
if (run == -1):
    print("player is not present")
else:
    print("{} score is {}".format(player, run))
    
```

```

* d1 = {1:"A", 2:"B", 3:"C"} # merge element list
d2 = {4:"x", 5:"y", 6:"z"}
d1.update(d2)
dg = {1:"a":4567, 'b':-2345}
d4 = {}
for i in (d1, d2, dg):
    d4.update(i)
print("After merge", d4)
    
```

```

* dg = {'a':4, 'b':5, 'c':9}
print(sum(dg.values())) # sum all atoms in a dictionary
    
```

```

* multiplication all atom in a dictionary.
dg = {'a':4, 'b':5, 'c':9}
print(sum(dg.values()))
x = 1
for i in dg.values():
    x *= i
print("multiplication of value", x)
    
```

```

* l = ['delhi':45, 'pune':37, 'mumbai':43]
d = {l[i]:l[i+1] for i in range(0, len(l)-1)}
print(d)
    
```


* write a program to remove duplicate from dictionary

```
student = { 'id': 1, 'name': 'Abhi', 'subject': ['cpp', 'java'],
            'grade': 49.95 },
            { 'id': 2, 'name': 'Raje', 'subject': ['cpp', 'java'],
            'grade': 98.23 },
            { 'id': 3, 'name': 'Raje', 'subject': ['cpp', 'java'],
            'grade': 98.23 }
```

```
result = {}
for i, j in student.items():
    if j not in result.values():
        result.update({i: j})
print('Result dictionary', result)
```

* Create a list as name of citys and temp of city and convert them into dictionary.

```
d = dict()
city = ['delhi', 'pune', 'mumbai']
temp = [45, 37, 43]
for i in range(len(city)):
    for j in range(len(temp)):
        d.update({city[i]: temp[j]})
print('dictionary', d)
```

* write a program to check minimum and maximum value from a dictionary

```
my_dict = { 'x': 500, 'y': 58.74, 'z': 560 }
key_max = max(my_dict.keys(), key=(lambda k: my_dict[k]))
key_min = min(my_dict.keys(), key=(lambda k: my_dict[k]))
print("maximum value:", my_dict[key_max])
print("minimum value:", my_dict[key_min])
```


* Write a program to add two different dictionary and display.

```

→ from collections import Counter
d1 = {'a': 100, 'b': 200, 'c': 300}
d2 = {'a': 300, 'b': 200, 'd': 400}
d = Counter(d1) + Counter(d2)
print(d)

```

* Write a python program to create and display all combination of later, selecting each later from a different key in a dictionary. A dictionary is already initialized.

```

→ import itertools
d = {'1': ['a', 'b'], '2': ['c', 'd']}
for combo in itertools.product(*[d[k] for k in sorted(d.keys())]):
    print(" ".join(combo))

```

* Write a program to create a dictionary from a string that tracks the count of letters from the string.

```

→ str1 = "Ashwadeep"
my_dict = {}
for letter in str1:
    my_dict[letter] = my_dict.get(letter, 0) + 1
print(my_dict)

```

* Write a program to find the top three items that is largest from a shop which represent in the form of dictionary.

```

→ from heapq import nlargest
d1 = {'item1': 45.50, 'item2': 35, 'item3': 41.30, 'item4': 55, 'item5': 24}
for name, value in nlargest(3, d1.items(), key=
    itemgetter(1)):
    print(name, value)

```


Page No. _____
Date ____/____/____

Function 8 - A function is a block of code which only runs when it is called. you can pass data, known as parameters into a function. A function can return data as a result.

A function is a block of code that perform a specific task.

* Create a function which prints sum of list variable bet? 1 to n

```
→ def sum_list(n):  
    s = 0  
    for i in range(n+1):  
        s += i  
    return(s)
```

```
n = int(input("Enter last position"))  
print(sum_list(n))
```

* Create a function is palindrom and checks input string is palindrom or not.

```
→ def ispalindrom(string):  
    left_pos = 0  
    right_pos = len(string) - 1  
    while right_pos >= left_pos:  
        if not string[left_pos] == string[right_pos]:  
            return false
```

```
        left_pos += 1
```

```
        right_pos -= 1
```

```
    return true
```

```
print(ispalindrom('a2a'))
```

* Create a function to find unique number list

```
→ def unique_list(l):  
    x = []  
    for a in l:  
        if a not in x:  
            x.append(a)  
    return x
```

```
print(unique_list([1,2,3,3,4,5,2,3]))
```


* Create a funⁿ to find max of three number
→ `def max3(a,b,c):`
 `if (a > b && a > c):`
 `return (a, "is a max")`
 `elif (b > a && b > c):`
 `return (b, "is a max")`
 `elif (c > b && c > a):`
 `return (c, "is a max")`
 `else:`
 `pass`
 `print (max3(5,10,12))`

* Create a simple calculator using user input define function that can add, sub, multi, divide using function

→ # define function.
`def add(x,y):`
 `"This is function adds two numbers"`
 `return x+y`
`def sub(x,y):`
 `return (x-y)`
`def multi(x,y):`
 `return (x*y)`
`def divide(x,y):`
 `return (x/y)`

* Take input from user

```
def trial():  
    num1 = int(input("Enter first number :"))  
    num2 = int(input("Enter second number :"))  
    print("welcome to perform arithmathical operation")  
    print("Select operation")  
    print("1. add")  
    print("2. sub")  
    print("3. multidivide")  
    print("4. divide")
```



```
choice = input("Enter your choice (1, 2, 3, 4):")
if choice == '1':
    print(num1, "+", num2, "=", add(num1, num2))
elif choice == '2':
    print(num1, "-", num2, "=", sub(num1, num2))
elif choice == '3':
    print(num1, "x" num2, "=", multi(num1, num2))
elif choice == '4':
    print(num1, "/", num2, "=", divide(num1, num2))
else:
    print("Invalid Input")
```

* Homework :-

- ① → Create a function to calculate the factorial of number.
- ② * Create a function that accept the string and calculate number uppercase letter and lower case letter.
- ③ * Create a function to print a list where the value are square and cube of the numbers betⁿ 1 to 30.
- ④ * Create a function to calculate area and perimeter of rectangle.

* lambda operator :- lambda operator is used to represent small, single line, anonymous function.
Syntax : lambda argu : expression.

* Create a funⁿ using lambda operator and print cube value.
 → `b = lambda a : a ** 3`
`print(b(7))`

* Create a funⁿ which prints table of a number which is input through the keyword.
 → `n = int(input("Enter any number"))`
`table = [lambda n: n * i`
`for i in range(1, 11):`
`print(table(n))`

* Create a funⁿ which check whether the string value start with or end with specific character then satisfied the condition.

→ `txt = input("Enter any text")`
`s = lambda txt: True if txt.startwith("A") and txt.endswith("p") else False`
`print(s(txt))`

* map function :- This function is used for mapping sequential element like list, tuple etc. It can be used with lambda without lambda.

syntax: `map(function definition, list of sequence)`

* Create a list of square element from the given sequence

`l1 = [1, 2, 3, 4, 5]`
`s1 = list(map(lambda x: x*x, l1))`
`print("sequence of list", s1)`

* create a list of function and display the value as per the given sequence.

→ `def add(x):`
`return (x+x)`
`def diff(x):`
`return (x-x)`
`def product(x):`
`return (x*x)`
`l1 = [add, diff, product]`
`for i in range(5):`
`n = list(map(lambda x: x(i), l1))`
`print(n)`

* Add three given list using map and function.

`l1 = [1, 2, 3], l2 = [4, 5, 6], l3 = [7, 8, 9]`
`l4 = list(map(lambda x, y, z: x+y+z, l1, l2, l3))`
`print(l4)`

* Create the tripple of all number of the given list of integers.

→ `l1 = [1, 2, 3]`

`l2 = list(map(lambda: x + 4 + 2, l1))`
`print(l2)`

* write a program to convert all characters in upper case and lower case and eliminate duplicate letter from a given sequence.

→ `letter = {'a', 'j', 'p', 'r', 'A', 'e', 'E', 'f'}`
`def change_case(x)`
 `return (x.upper() + x.lower())`
`test = map(change_case, letter)`
`print(set(test))`

* Create two lists and find total and difference between the two list using map.

→ `num1 = [10, 20, 30]`
`num2 = [4, 5, 6]`
`def add_diff(n1, n2):`
 `return (n1 + n2, n1 - n2)`
`result = map(add_diff, num1, num2)`
`print(result)`

* Filter :- filter is a built-in function to find the specified condition and filter out all the specified condition elements of a sequence, for which function return true.
Syntax: `filter(function definition, sequence)`
Ex:

`num = range(1, 101)`

`even = list(filter(lambda x: x % 2 == 0, num))`
`print(even)`

* using filter iterate each element for the sequence and print if it vowel then true either false.

```
→ str = ['a', 'e', 'i', 'o', 'u', 'y', 'A', 'E', 'I', 'O', 'U', 'Y']
def checks(str):
    vowel = ['a', 'e', 'i', 'o', 'u']
    if str in vowel:
        return True
    else:
        return False
```

```
testlist = filter(checks, str)
print(testlist)
```

* create a list and print only divisible 5 and 7

```
→ l1 = [1, 2, 5, 7, 10, 14, 15, 21]
num = list(filter(lambda x: x % 5 == 0 and x % 7 == 0, l1))
print(num)
```

* Reduce function :- It is really usefull for performing some computation on list returning the result it applies a rolling computation to sequential pairs of values.

syntax: reduce [function definition, sequence]

* create a function and find factorial of a given number

```
import functools as f
l1 = range(1, 6)
fact = f.reduce(lambda x, y: x * y, l1)
print("factorial value", fact)
```

* write a program to find Fibonacci series using reduce fun.

```
→ from functools import reduce
fib_series = lambda n: reduce(lambda x, y: x[1] + x[0], range(n-2), [0, 1])
print("Fibonacci series upto 2:")
print("fibonacci series (2)")
```